

RECURSIVITE - Exercices

Exercice 1

On considère la fonction Python `mystere` suivante qui prend un tableau d'entiers `t` et un entier `i` en paramètres.

```
def mystere(t,i):
    if i == 0:
        return 0
    else:
        return t[i-1] + mystere(t,i-1)
```

- Quel est le résultat de `mystere([10,9,11,42],3)` ? Dessiner l'arbre des appels correspondants.
- En supposant que `i` est toujours compris entre 0 et `len(t)`, expliquer en une phrase ce que calcul la fonction `mystere(t,i)`.

Exercice 2

Expliquer pourquoi les fonctions récursives f , g et h suivantes ne sont pas correctement définies.

$$f(n) = \begin{cases} 1 & \text{si } n = 0 \\ n + f(n+1) & \text{si } n > 0 \end{cases}$$

$$g(n) = \begin{cases} 1 & \text{si } n = 0 \\ n + g(n-2) & \text{si } n > 0 \end{cases}$$

$$h(n) = \begin{cases} 1 & \text{si } n = 0 \\ h(n-1) & \text{si } n > 1 \end{cases}$$

Même question avec la fonction a suivante :

$$a(n) = \begin{cases} 1 & \text{si } n = 0 \\ 2 \times a(2n) & \text{si } n \text{ est impair} \\ 1 + a(n-1) & \text{si } n \text{ est pair} \end{cases}$$

Exercice 3

On définit la fonction python ci-dessous dans laquelle `L` est une liste.

```
def r(x,a,b,L):
    if a > b:
        return "fini "
    t = (a+b)//2
    if x == L[t]:
        return t
    if x < L[t]:
        return r(x,a,t-1,L)
    else:
        return r(x,t+1,b,L)
```

Décrire sur papier les étapes lors de l'appel ci-dessous. Quel est le nom de l'algorithme utilisé ici ?

```
L = [2,3,4,7,11,15,17]
print(r(4,0,len(L)-1,L))
```

Exercise 4

On considère la fonction récursive `mystere` qui prend en argument un nombre entier positif et dont le code est indiqué ci-contre.

```
def mystere(x):  
    if x == 0:  
        return 2  
    elif x == 1:  
        return 3  
    elif x == 2:  
        return 5  
    else:  
        return mystere(x-3) * mystere(x-2)
```

- 1) Encadrer les lignes de code correspondant au(x) cas de base.
- 2) Donner la formule de récursivité correspondant au cas général.
- 3) Donner l'arbre récursif des appels lors de l'appel principal `mystere(10)`.
On pourra utiliser $m(x)$ comme abréviation de `mystere(x)`.
- 4) Sur cet arbre récursif, entourer deux sous-arbres identiques qui montrent qu'on répète certains calculs plusieurs fois lors de l'appel à `mystere(10)`.
- 5) Donner la valeur renvoyée par `mystere(10)`.

Exercise 5

Voici trois fonctions « mystère ».

Pour chacune d'entre elles, indiquer ce que représente la valeur renvoyée en fonction de la variable passée en argument.

Code Python

```
def mystery1(x : int) -> int :  
    """ x is a positive integer """  
    if x == 0 :  
        return 1  
    else :  
        return 2 * mystery1(x - 1)
```

Code Python

```
def mystery2(lst : list) -> int :  
    """ lst is a list of int and can be empty """  
    if lst != [] :  
        return lst[0] + mystery2(lst[1 :])  
    else :  
        return 0
```

Code Python

```
def mystery3(lst : list) -> bool :  
    """ lst is a list of int """  
    if len(lst) > 1 :  
        if lst[0] > lst[1] :  
            return False  
        else :  
            return mystery3(lst[1 :])  
    else :  
        return True
```